

CS/EE 147 Final Project Proposal

CUDA-Accelerated Ray Tracer with Multi-GPU Rendering

Spring 2026 | Team Members: Chris Antony, Kaushik Vada

1. Required Libraries and Frameworks

- CUDA — primary GPU compute framework
- NVIDIA CUDA Math Library (libm) — vector and floating-point operations
- stb_image_write.h — lightweight single-header library for PNG image output
- OpenMP — CPU baseline multithreading for performance comparison

2. Potential Risks and Technical Challenges

- BVH construction on GPU: Building the bounding volume hierarchy efficiently in parallel is non-trivial. We may fall back to CPU-side BVH construction with GPU traversal if needed.
- Multi-GPU coordination: Splitting the framebuffer across two A30s and stitching the output requires careful cross-device memory management. This will be attempted after single-GPU is stable.
- Warp divergence: Ray tracing causes inherent thread divergence (rays terminate at different depths, hit different geometry). We will profile and mitigate if it becomes a bottleneck.
- Cluster setup: Confirming CUDA toolkit availability and GPU allocation on UCR machines may require early sprint time.

3. Project Plan and Outline

The goal is a CUDA ray tracer that renders a 3D scene (spheres, boxes, floor plane, point lights, shadows) and outputs a high-resolution image. Each pixel maps to one GPU thread in a 2D thread grid, making it a natural fit for massive parallelism. We will layer in two key optimizations and benchmark across four configurations: CPU single-threaded, CUDA naive, CUDA with BVH, and dual-GPU.

Week	Goals
Week 1 (Apr 30 - May 7)	Cluster environment setup; implement CPU baseline ray tracer with spheres, point lights, and shadows
Week 2 (May 8 - May 14)	Port to CUDA: naive 1-thread-per-pixel kernel; verify correctness against CPU output
Week 3 (May 15 - May 21)	Implement BVH acceleration structure; benchmark naive vs. BVH across scene sizes
Week 4 (May 22 - May 28)	Attempt dual-GPU rendering (split framebuffer across both A30s); collect all benchmark data

Week 5 (May 29 - Jun 1)	Report writeup, code cleanup, presentation preparation and demo
-------------------------	---

4. References

- Peter Shirley, Ray Tracing in One Weekend (scene setup and CPU baseline):
<https://raytracing.github.io/>
- NVIDIA CUDA C Programming Guide:
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- Tero Karras, Thinking Parallel Part II: Tree Traversal on the GPU (BVH on GPU):
<https://developer.nvidia.com/blog/thinking-parallel-part-ii-tree-traversal-gpu/>
- NVIDIA Multi-GPU Programming:
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#multi-device-system>